

Universidad Tecnológica Nacional

Tecnicatura en
Programación a
Distancia
Trabajo Práctico
Integrador en
Programación.

Integrantes:
Ariel Villegas
Marilyn Ruiz

Comisión: 7

Tema:

"Gestión de Datos de Países en
Python: filtros, ordenamientos y
estadísticas"

Fecha: 17/06/26

```
font-size: 1.2em;
background: url(Cyber.jpg);
background-size: cover;

.box{
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%,
    width: 400px;
    padding: 40px;
    background: rgba(0, 0, 0,
    box-sizing: border-box;
    box-shadow: 0 15px 25px rgba(0, 0, 0, 0.5);
    border-radius: 10px;
}
```

```
.box h2{
    margin: 0 0 30px;
    padding: 0;
    color: #fff;
    text-align: center;
}
```

```
.box h3{
    margin: 0 0 10px;
    padding: 0;
    color: #fff;
    text-align: center;
}
```

```
.box .inputBox{
    position: relative;
}
```

ÍNDICE

1. Introducción.....	2
2. Objetivo del Trabajo.....	2
3. Marco Teórico.....	3
3.1 Listas	
3.2 Diccionarios	
3.3 Funciones	
3.4 Estructuras Condicionales	
3.5 Estructuras Repetitivas	
3.6 Ordenamientos	
3.7 Estadísticas Básicas	
3.8 Archivos CSV	
4. Diseño del Caso Práctico.....	5
5. Metodología Utilizada.....	9
6. Resultados Obtenidos.....	9
7. Dificultades Encontradas.....	10
8. Conclusiones.....	10
9. Bibliografía.....	10
10.Repositorio GitHub	
11.Video Demostración	

1. INTRODUCCIÓN

El presente trabajo práctico consiste en el desarrollo de una aplicación en Python para la gestión de información de países. El sistema permite cargar información desde un archivo CSV, realizar búsquedas, filtros, ordenamientos y obtener estadísticas relevantes sobre los datos almacenados.

El proyecto fue desarrollado utilizando estructuras de datos fundamentales de Python, aplicando conceptos vistos durante la cursada de Programación I.

2. OBJETIVO DEL TRABAJO

Desarrollar una aplicación de consola que permita gestionar información de países mediante la utilización de listas, diccionarios, funciones, estructuras condicionales, estructuras repetitivas y archivos CSV.

Asimismo, se busca aplicar técnicas de búsqueda, filtrado, ordenamiento y generación de estadísticas para procesar información de manera eficiente.

3. MARCO TEÓRICO

3.1 LISTAS

Las listas son estructuras de datos que permiten almacenar múltiples elementos dentro de una misma variable. Son mutables y permiten agregar, eliminar o modificar elementos durante la ejecución del programa.

En este proyecto se utiliza una lista para almacenar todos los países cargados desde el archivo CSV.

Fuente:

Python Documentation.

<https://docs.python.org/3/tutorial/datastructures.html>

3.2 DICCIONARIOS

Los diccionarios almacenan información mediante pares clave-valor.

Cada país es representado mediante un diccionario con las claves:

- nombre
- poblacion
- superficie
- continente

Fuente:

Python Documentation.

<https://docs.python.org/3/tutorial/datastructures.html#dictionaries>

3.3 FUNCIONES

Las funciones permiten dividir el programa en bloques reutilizables de código.

En el sistema se implementaron funciones independientes para carga de datos, búsquedas, filtros, ordenamientos y estadísticas.

Fuente:

Python Documentation.

<https://docs.python.org/3/tutorial/controlflow.html#defining-functions>

3.4 ESTRUCTURAS CONDICIONALES

Las estructuras condicionales permiten ejecutar diferentes acciones dependiendo de una condición determinada.

En este proyecto se utilizan instrucciones if, elif y else para validar datos ingresados por el usuario y controlar las opciones seleccionadas en el menú.

Fuente:

Python Documentation.

<https://docs.python.org/3/tutorial/controlflow.html>

3.5 ESTRUCTURAS REPETITIVAS

Las estructuras repetitivas permiten ejecutar instrucciones múltiples veces.

En este trabajo se utilizan ciclos while y for para recorrer listas de países y mantener activo el menú principal.

Fuente:

Python Documentation.

<https://docs.python.org/3/tutorial/controlflow.html>

3.6 ORDENAMIENTOS

Los ordenamientos permiten organizar la información según un criterio determinado.

El sistema permite ordenar países por nombre, población y superficie utilizando la función sorted().

Fuente:

Python Documentation.

<https://docs.python.org/3/library/functions.html#sorted>

3.7 ESTADÍSTICAS BÁSICAS

Las estadísticas básicas permiten obtener información relevante a partir de un conjunto de datos.

En este proyecto se calculan:

- País con mayor población.
- País con menor población.
- Promedio de población.
- Promedio de superficie.
- Cantidad de países por continente.

Fuente:

Downey, A. Think Python.

3.8 ARCHIVOS CSV

Los archivos CSV permiten almacenar datos tabulares separados por comas.

El sistema utiliza el módulo csv de Python para leer la información inicial de los países.

Fuente:

Python Documentation.

<https://docs.python.org/3/library/csv.html>

4. DISEÑO DEL CASO PRÁCTICO

Flujo General:



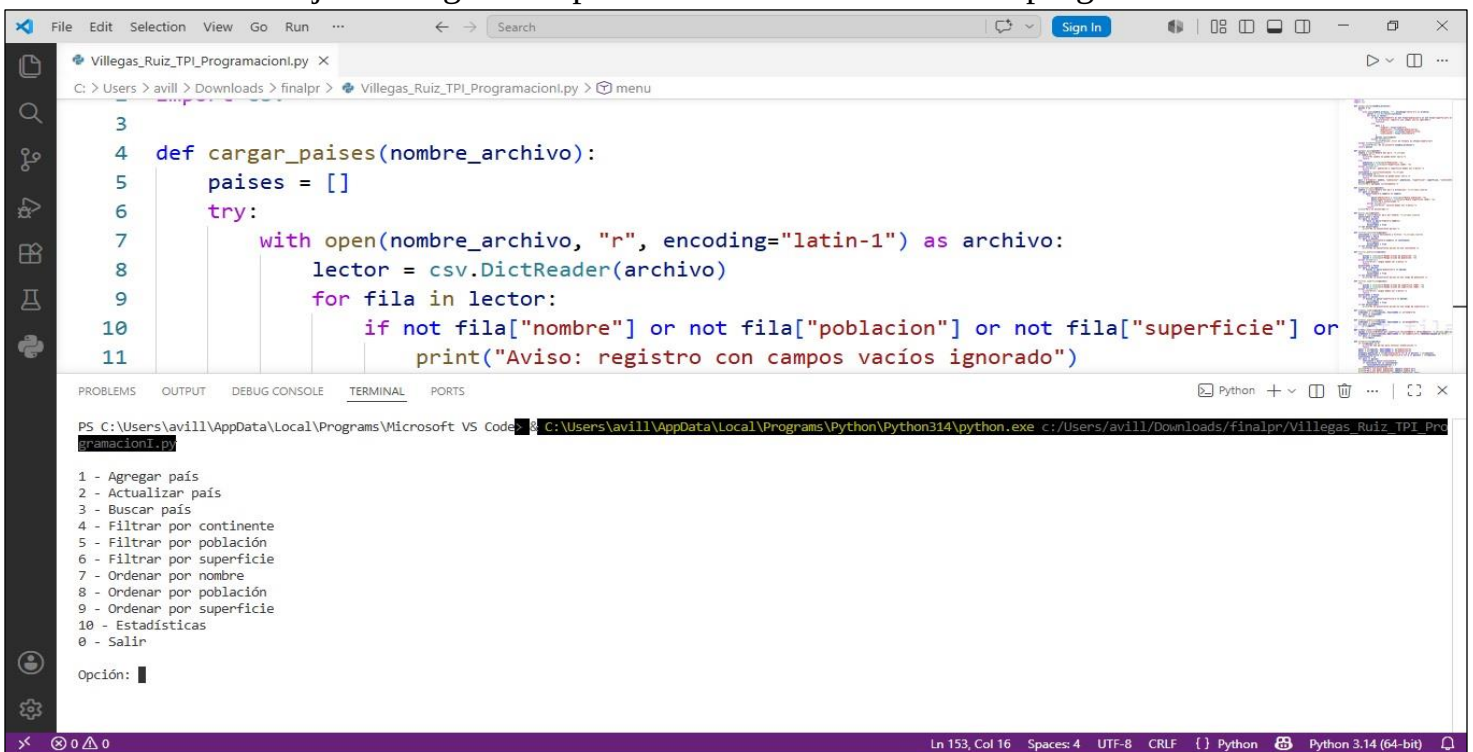
Estructura de datos utilizada:

Lista de diccionarios.

Ejemplo:

```
{  
"nombre": "Argentina",  
"poblacion": 45376763,  
"superficie": 2780400,  
"continente": "América"  
}
```

A continuación se adjuntan algunas capturas del funcionamiento del programa:



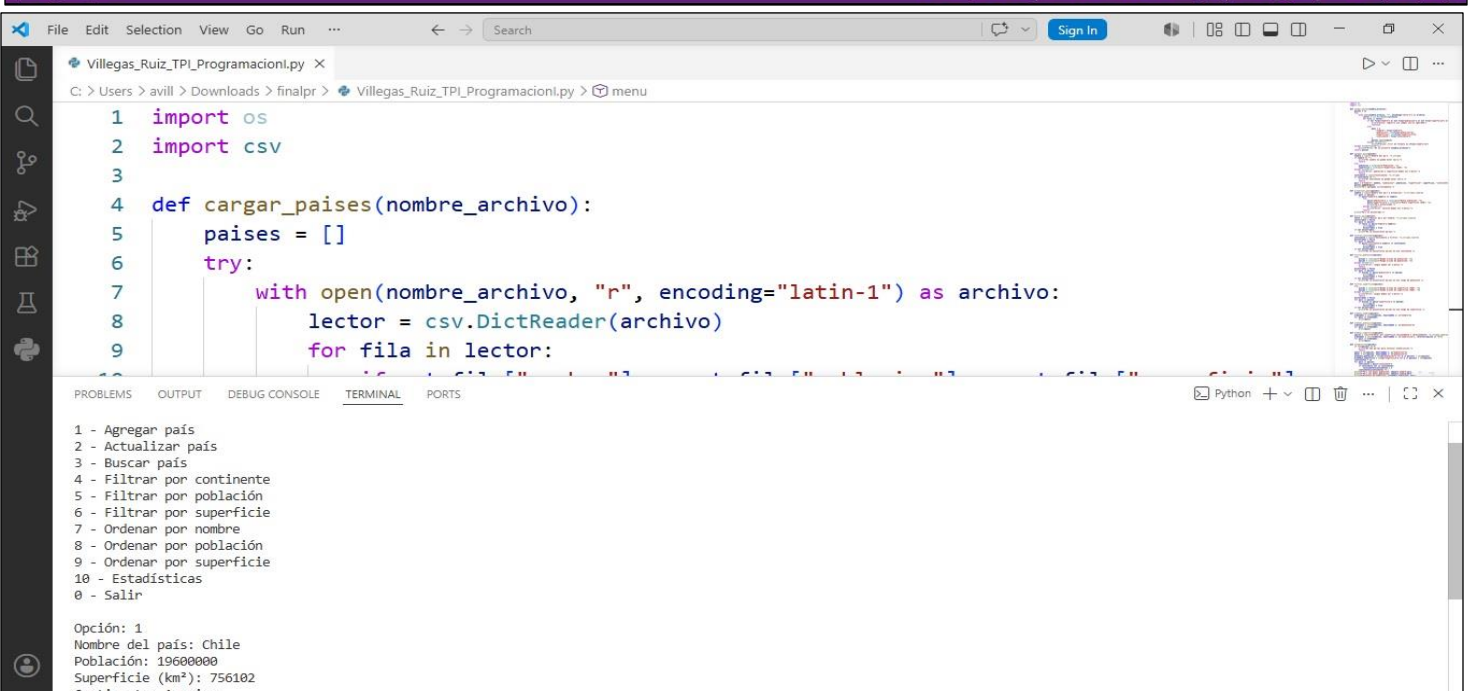
The screenshot shows a VS Code editor window with a Python file named 'Villegas_Ruiz_TPI_ProgramacionI.py'. The code defines a function 'cargar_paises' that reads a CSV file and prints a warning for empty fields. The terminal output shows a menu with options 1-10 and the user has selected option 1.

```
3  
4 def cargar_paises(nombre_archivo):  
5     paises = []  
6     try:  
7         with open(nombre_archivo, "r", encoding="latin-1") as archivo:  
8             lector = csv.DictReader(archivo)  
9             for fila in lector:  
10                 if not fila["nombre"] or not fila["poblacion"] or not fila["superficie"] or  
11                     print("Aviso: registro con campos vacíos ignorado")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\avill\AppData\Local\Programs\Microsoft VS Code> C:\Users\avill\AppData\Local\Programs\Python\Python314\python.exe c:/Users/avill/Downloads/finalpr/Villegas_Ruiz_TPI_ProgramacionI.py

```
1 - Agregar país  
2 - Actualizar país  
3 - Buscar país  
4 - Filtrar por continente  
5 - Filtrar por población  
6 - Filtrar por superficie  
7 - Ordenar por nombre  
8 - Ordenar por población  
9 - Ordenar por superficie  
10 - Estadísticas  
0 - Salir  
Opción: 1
```



The screenshot shows the same VS Code editor window, but now the code includes imports for 'os' and 'csv'. The terminal output shows the user has selected option 1, and the program has displayed the details for Chile.

```
1 import os  
2 import csv  
3  
4 def cargar_paises(nombre_archivo):  
5     paises = []  
6     try:  
7         with open(nombre_archivo, "r", encoding="latin-1") as archivo:  
8             lector = csv.DictReader(archivo)  
9             for fila in lector:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
1 - Agregar país  
2 - Actualizar país  
3 - Buscar país  
4 - Filtrar por continente  
5 - Filtrar por población  
6 - Filtrar por superficie  
7 - Ordenar por nombre  
8 - Ordenar por población  
9 - Ordenar por superficie  
10 - Estadísticas  
0 - Salir  
Opción: 1  
Nombre del país: Chile  
Población: 19600000  
Superficie (km²): 756102
```

```
File Edit Selection View Go Run ... Search Sign In
Villegas_Ruiz_TPI_Programacion1.py X
C:\Users> avill > Downloads > finalpr > Villegas_Ruiz_TPI_Programacion1.py > menu

1 import os
2 import csv
3
4 def cargar_paises(nombre_archivo):
5     paises = []
6     try:
7         with open(nombre_archivo, "r", encoding="latin-1") as archivo:
8             lector = csv.DictReader(archivo)
9             for fila in lector:
10                 paises.append(fila)
11     except:
12         pass
13
14 # Función para mostrar el menú de opciones
15 def mostrar_menu():
16     print("1 - Agregar país")
17     print("2 - Actualizar país")
18     print("3 - Buscar país")
19     print("4 - Filtrar por continente")
20     print("5 - Filtrar por población")
21     print("6 - Filtrar por superficie")
22     print("7 - Ordenar por nombre")
23     print("8 - Ordenar por población")
24     print("9 - Ordenar por superficie")
25     print("10 - Estadísticas")
26     print("0 - Salir")
27
28 # Función para buscar un país por nombre
29 def buscar_pais(nombre):
30     paises = cargar_paises('paises.csv')
31     for pais in paises:
32         if pais['nombre'] == nombre:
33             return pais
34     return None
35
36 # Función para filtrar países por continente
37 def filtrar_por_continente(continente):
38     paises = cargar_paises('paises.csv')
39     return [pais for pais in paises if pais['continente'] == continente]
40
41 # Función para filtrar países por población
42 def filtrar_por_poblacion(poblacion):
43     paises = cargar_paises('paises.csv')
44     return [pais for pais in paises if pais['poblacion'] > poblacion]
45
46 # Función para filtrar países por superficie
47 def filtrar_por_superficie(superficie):
48     paises = cargar_paises('paises.csv')
49     return [pais for pais in paises if pais['superficie'] > superficie]
50
51 # Función para ordenar países por nombre
52 def ordenar_por_nombre():
53     paises = cargar_paises('paises.csv')
54     return sorted(paises, key=lambda x: x['nombre'])
55
56 # Función para ordenar países por población
57 def ordenar_por_poblacion():
58     paises = cargar_paises('paises.csv')
59     return sorted(paises, key=lambda x: x['poblacion'])
60
61 # Función para ordenar países por superficie
62 def ordenar_por_superficie():
63     paises = cargar_paises('paises.csv')
64     return sorted(paises, key=lambda x: x['superficie'])
65
66 # Función para mostrar estadísticas
67 def mostrar_estadisticas():
68     paises = cargar_paises('paises.csv')
69     print("Total de países: ", len(paises))
70     print("Países por continente:")
71     for continente in ['America', 'Europa', 'Asia', 'Africa', 'Oceania']:
72         paises_continente = filtrar_por_continente(continente)
73         print(f"{continente}: {len(paises_continente)} países")
74
75 # Programa principal
76 if __name__ == '__main__':
77     mostrar_menu()
78     opcion = input("Opción: ")
79     if opcion == '0':
80         exit()
81     elif opcion == '1':
82         nombre = input("Nombre del país a agregar: ")
83         pais = {'nombre': nombre, 'poblacion': 0, 'superficie': 0, 'continente': ''}
84         paises = cargar_paises('paises.csv')
85         paises.append(pais)
86         with open('paises.csv', 'w', encoding='latin-1') as archivo:
87             escritor = csv.DictWriter(archivo, fieldnames=paises[0].keys())
88             escritor.writeheader()
89             for pais in paises:
90                 escritor.writerow(pais)
91     elif opcion == '2':
92         nombre = input("Nombre del país a actualizar: ")
93         pais = buscar_pais(nombre)
94         if pais:
95             poblacion = input("Población: ")
96             superficie = input("Superficie: ")
97             pais['poblacion'] = poblacion
98             pais['superficie'] = superficie
99             paises = cargar_paises('paises.csv')
100            paises[paises.index(pais)] = pais
101            with open('paises.csv', 'w', encoding='latin-1') as archivo:
102                escritor = csv.DictWriter(archivo, fieldnames=paises[0].keys())
103                escritor.writeheader()
104                for pais in paises:
105                    escritor.writerow(pais)
106        else:
107            print("País no encontrado.")
108    elif opcion == '3':
109        nombre = input("Nombre del país a buscar: ")
110        pais = buscar_pais(nombre)
111        if pais:
112            print(pais)
113        else:
114            print("País no encontrado.")
115    elif opcion == '4':
116        continente = input("Continente a filtrar: ")
117        paises = filtrar_por_continente(continente)
118        print(paises)
119    elif opcion == '5':
120        poblacion = input("Población a filtrar: ")
121        paises = filtrar_por_poblacion(poblacion)
122        print(paises)
123    elif opcion == '6':
124        superficie = input("Superficie a filtrar: ")
125        paises = filtrar_por_superficie(superficie)
126        print(paises)
127    elif opcion == '7':
128        paises = ordenar_por_nombre()
129        print(paises)
130    elif opcion == '8':
131        paises = ordenar_por_poblacion()
132        print(paises)
133    elif opcion == '9':
134        paises = ordenar_por_superficie()
135        print(paises)
136    elif opcion == '10':
137        mostrar_estadisticas()
138    else:
139        print("Opción no válida.")
140
141 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
0 - Salir
Opción: 3
Buscar país por nombre: arg
{'nombre': 'Argentina', 'poblacion': 45376763, 'superficie': 2780400, 'continente': 'America'}
1 - Agregar país
2 - Actualizar país
3 - Buscar país
4 - Filtrar por continente
5 - Filtrar por población
6 - Filtrar por superficie
7 - Ordenar por nombre
8 - Ordenar por población
9 - Ordenar por superficie
10 - Estadísticas
0 - Salir
Opción: 
```

```
File Edit Selection View Go Run ... Search Sign In
Villegas_Ruiz_TPI_Programacion1.py X
C:\Users> avill > Downloads > finalpr > Villegas_Ruiz_TPI_Programacion1.py > menu

1 import os
2 import csv
3
4 def cargar_paises(nombre_archivo):
5     paises = []
6     try:
7         with open(nombre_archivo, "r", encoding="latin-1") as archivo:
8             lector = csv.DictReader(archivo)
9             for fila in lector:
10                 paises.append(fila)
11     except:
12         pass
13
14 # Función para mostrar el menú de opciones
15 def mostrar_menu():
16     print("1 - Agregar país")
17     print("2 - Actualizar país")
18     print("3 - Buscar país")
19     print("4 - Filtrar por continente")
20     print("5 - Filtrar por población")
21     print("6 - Filtrar por superficie")
22     print("7 - Ordenar por nombre")
23     print("8 - Ordenar por población")
24     print("9 - Ordenar por superficie")
25     print("10 - Estadísticas")
26     print("0 - Salir")
27
28 # Función para buscar un país por nombre
29 def buscar_pais(nombre):
30     paises = cargar_paises('paises.csv')
31     for pais in paises:
32         if pais['nombre'] == nombre:
33             return pais
34     return None
35
36 # Función para filtrar países por continente
37 def filtrar_por_continente(continente):
38     paises = cargar_paises('paises.csv')
39     return [pais for pais in paises if pais['continente'] == continente]
40
41 # Función para filtrar países por población
42 def filtrar_por_poblacion(poblacion):
43     paises = cargar_paises('paises.csv')
44     return [pais for pais in paises if pais['poblacion'] > poblacion]
45
46 # Función para filtrar países por superficie
47 def filtrar_por_superficie(superficie):
48     paises = cargar_paises('paises.csv')
49     return [pais for pais in paises if pais['superficie'] > superficie]
50
51 # Función para ordenar países por nombre
52 def ordenar_por_nombre():
53     paises = cargar_paises('paises.csv')
54     return sorted(paises, key=lambda x: x['nombre'])
55
56 # Función para ordenar países por población
57 def ordenar_por_poblacion():
58     paises = cargar_paises('paises.csv')
59     return sorted(paises, key=lambda x: x['poblacion'])
60
61 # Función para ordenar países por superficie
62 def ordenar_por_superficie():
63     paises = cargar_paises('paises.csv')
64     return sorted(paises, key=lambda x: x['superficie'])
65
66 # Función para mostrar estadísticas
67 def mostrar_estadisticas():
68     paises = cargar_paises('paises.csv')
69     print("Total de países: ", len(paises))
70     print("Países por continente:")
71     for continente in ['America', 'Europa', 'Asia', 'Africa', 'Oceania']:
72         paises_continente = filtrar_por_continente(continente)
73         print(f"{continente}: {len(paises_continente)} países")
74
75 # Programa principal
76 if __name__ == '__main__':
77     mostrar_menu()
78     opcion = input("Opción: ")
79     if opcion == '0':
80         exit()
81     elif opcion == '1':
82         nombre = input("Nombre del país a agregar: ")
83         pais = {'nombre': nombre, 'poblacion': 0, 'superficie': 0, 'continente': ''}
84         paises = cargar_paises('paises.csv')
85         paises.append(pais)
86         with open('paises.csv', 'w', encoding='latin-1') as archivo:
87             escritor = csv.DictWriter(archivo, fieldnames=paises[0].keys())
88             escritor.writeheader()
89             for pais in paises:
90                 escritor.writerow(pais)
91     elif opcion == '2':
92         nombre = input("Nombre del país a actualizar: ")
93         pais = buscar_pais(nombre)
94         if pais:
95             poblacion = input("Población: ")
96             superficie = input("Superficie: ")
97             pais['poblacion'] = poblacion
98             pais['superficie'] = superficie
99             paises = cargar_paises('paises.csv')
100            paises[paises.index(pais)] = pais
101            with open('paises.csv', 'w', encoding='latin-1') as archivo:
102                escritor = csv.DictWriter(archivo, fieldnames=paises[0].keys())
103                escritor.writeheader()
104                for pais in paises:
105                    escritor.writerow(pais)
106        else:
107            print("País no encontrado.")
108    elif opcion == '3':
109        nombre = input("Nombre del país a buscar: ")
110        pais = buscar_pais(nombre)
111        if pais:
112            print(pais)
113        else:
114            print("País no encontrado.")
115    elif opcion == '4':
116        continente = input("Continente a filtrar: ")
117        paises = filtrar_por_continente(continente)
118        print(paises)
119    elif opcion == '5':
120        poblacion = input("Población a filtrar: ")
121        paises = filtrar_por_poblacion(poblacion)
122        print(paises)
123    elif opcion == '6':
124        superficie = input("Superficie a filtrar: ")
125        paises = filtrar_por_superficie(superficie)
126        print(paises)
127    elif opcion == '7':
128        paises = ordenar_por_nombre()
129        print(paises)
130    elif opcion == '8':
131        paises = ordenar_por_poblacion()
132        print(paises)
133    elif opcion == '9':
134        paises = ordenar_por_superficie()
135        print(paises)
136    elif opcion == '10':
137        mostrar_estadisticas()
138    else:
139        print("Opción no válida.")
140
141 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Opción: 4
Continente a filtrar: Europa
{'nombre': 'Alemania', 'poblacion': 83149300, 'superficie': 357022, 'continente': 'Europa'}
{'nombre': 'Francia', 'poblacion': 67430000, 'superficie': 543965, 'continente': 'Europa'}
{'nombre': 'Italia', 'poblacion': 59270000, 'superficie': 301340, 'continente': 'Europa'}
1 - Agregar país
2 - Actualizar país
3 - Buscar país
4 - Filtrar por continente
5 - Filtrar por población
6 - Filtrar por superficie
7 - Ordenar por nombre
8 - Ordenar por población
9 - Ordenar por superficie
10 - Estadísticas
0 - Salir
Opción: 
```



The screenshot shows a Python IDE with a file named 'Villegas_Ruiz_TPI_ProgramacionI.py'. The code defines a function 'cargar_paises' that takes a file path as an argument. It uses 'open' with 'r' mode and 'latin-1' encoding to read the file, and 'csv.DictReader' to parse the CSV data. A menu is printed to the terminal, allowing the user to select a sorting criterion (by name, population, or area) or to exit. The terminal output shows the menu and the result of selecting option 5 (sort by area), displaying a list of countries with their population and area.

```
1 import os
2 import csv
3
4 def cargar_paises(nombre_archivo):
5     paises = []
6     try:
7         with open(nombre_archivo, "r", encoding="latin-1") as archivo:
8             lector = csv.DictReader(archivo)
9             for fila in lector:
10                 paises.append(fila)
11
12     except FileNotFoundError:
13         print(f"El archivo {nombre_archivo} no existe.")
14
15 if __name__ == '__main__':
16     menu()
17     cargar_paises('paises.csv')
18     menu()
19     cargar_paises('paises.csv')
20     menu()
21     cargar_paises('paises.csv')
22     menu()
23     cargar_paises('paises.csv')
24     menu()
25     cargar_paises('paises.csv')
26     menu()
27     cargar_paises('paises.csv')
28     menu()
29     cargar_paises('paises.csv')
30     menu()
31     cargar_paises('paises.csv')
32     menu()
33     cargar_paises('paises.csv')
34     menu()
35     cargar_paises('paises.csv')
36     menu()
37     cargar_paises('paises.csv')
38     menu()
39     cargar_paises('paises.csv')
40     menu()
41     cargar_paises('paises.csv')
42     menu()
43     cargar_paises('paises.csv')
44     menu()
45     cargar_paises('paises.csv')
46     menu()
47     cargar_paises('paises.csv')
48     menu()
49     cargar_paises('paises.csv')
50     menu()
51     cargar_paises('paises.csv')
52     menu()
53     cargar_paises('paises.csv')
54     menu()
55     cargar_paises('paises.csv')
56     menu()
57     cargar_paises('paises.csv')
58     menu()
59     cargar_paises('paises.csv')
60     menu()
61     cargar_paises('paises.csv')
62     menu()
63     cargar_paises('paises.csv')
64     menu()
65     cargar_paises('paises.csv')
66     menu()
67     cargar_paises('paises.csv')
68     menu()
69     cargar_paises('paises.csv')
70     menu()
71     cargar_paises('paises.csv')
72     menu()
73     cargar_paises('paises.csv')
74     menu()
75     cargar_paises('paises.csv')
76     menu()
77     cargar_paises('paises.csv')
78     menu()
79     cargar_paises('paises.csv')
80     menu()
81     cargar_paises('paises.csv')
82     menu()
83     cargar_paises('paises.csv')
84     menu()
85     cargar_paises('paises.csv')
86     menu()
87     cargar_paises('paises.csv')
88     menu()
89     cargar_paises('paises.csv')
90     menu()
91     cargar_paises('paises.csv')
92     menu()
93     cargar_paises('paises.csv')
94     menu()
95     cargar_paises('paises.csv')
96     menu()
97     cargar_paises('paises.csv')
98     menu()
99     cargar_paises('paises.csv')
100    menu()
101    cargar_paises('paises.csv')
102    menu()
103    cargar_paises('paises.csv')
104    menu()
105    cargar_paises('paises.csv')
106    menu()
107    cargar_paises('paises.csv')
108    menu()
109    cargar_paises('paises.csv')
110    menu()
111    cargar_paises('paises.csv')
112    menu()
113    cargar_paises('paises.csv')
114    menu()
115    cargar_paises('paises.csv')
116    menu()
117    cargar_paises('paises.csv')
118    menu()
119    cargar_paises('paises.csv')
120    menu()
121    cargar_paises('paises.csv')
122    menu()
123    cargar_paises('paises.csv')
124    menu()
125    cargar_paises('paises.csv')
126    menu()
127    cargar_paises('paises.csv')
128    menu()
129    cargar_paises('paises.csv')
130    menu()
131    cargar_paises('paises.csv')
132    menu()
133    cargar_paises('paises.csv')
134    menu()
135    cargar_paises('paises.csv')
136    menu()
137    cargar_paises('paises.csv')
138    menu()
139    cargar_paises('paises.csv')
140    menu()
141    cargar_paises('paises.csv')
142    menu()
143    cargar_paises('paises.csv')
144    menu()
145    cargar_paises('paises.csv')
146    menu()
147    cargar_paises('paises.csv')
148    menu()
149    cargar_paises('paises.csv')
150    menu()
151    cargar_paises('paises.csv')
152    menu()
153    cargar_paises('paises.csv')
154    menu()
155    cargar_paises('paises.csv')
156    menu()
157    cargar_paises('paises.csv')
158    menu()
159    cargar_paises('paises.csv')
160    menu()
161    cargar_paises('paises.csv')
162    menu()
163    cargar_paises('paises.csv')
164    menu()
165    cargar_paises('paises.csv')
166    menu()
167    cargar_paises('paises.csv')
168    menu()
169    cargar_paises('paises.csv')
170    menu()
171    cargar_paises('paises.csv')
172    menu()
173    cargar_paises('paises.csv')
174    menu()
175    cargar_paises('paises.csv')
176    menu()
177    cargar_paises('paises.csv')
178    menu()
179    cargar_paises('paises.csv')
180    menu()
181    cargar_paises('paises.csv')
182    menu()
183    cargar_paises('paises.csv')
184    menu()
185    cargar_paises('paises.csv')
186    menu()
187    cargar_paises('paises.csv')
188    menu()
189    cargar_paises('paises.csv')
190    menu()
191    cargar_paises('paises.csv')
192    menu()
193    cargar_paises('paises.csv')
194    menu()
195    cargar_paises('paises.csv')
196    menu()
197    cargar_paises('paises.csv')
198    menu()
199    cargar_paises('paises.csv')
200    menu()
201    cargar_paises('paises.csv')
202    menu()
203    cargar_paises('paises.csv')
204    menu()
205    cargar_paises('paises.csv')
206    menu()
207    cargar_paises('paises.csv')
208    menu()
209    cargar_paises('paises.csv')
210    menu()
211    cargar_paises('paises.csv')
212    menu()
213    cargar_paises('paises.csv')
214    menu()
215    cargar_paises('paises.csv')
216    menu()
217    cargar_paises('paises.csv')
218    menu()
219    cargar_paises('paises.csv')
220    menu()
221    cargar_paises('paises.csv')
222    menu()
223    cargar_paises('paises.csv')
224    menu()
225    cargar_paises('paises.csv')
226    menu()
227    cargar_paises('paises.csv')
228    menu()
229    cargar_paises('paises.csv')
230    menu()
231    cargar_paises('paises.csv')
232    menu()
233    cargar_paises('paises.csv')
234    menu()
235    cargar_paises('paises.csv')
236    menu()
237    cargar_paises('paises.csv')
238    menu()
239    cargar_paises('paises.csv')
240    menu()
241    cargar_paises('paises.csv')
242    menu()
243    cargar_paises('paises.csv')
244    menu()
245    cargar_paises('paises.csv')
246    menu()
247    cargar_paises('paises.csv')
248    menu()
249    cargar_paises('paises.csv')
250    menu()
251    cargar_paises('paises.csv')
252    menu()
253    cargar_paises('paises.csv')
254    menu()
255    cargar_paises('paises.csv')
256    menu()
257    cargar_paises('paises.csv')
258    menu()
259    cargar_paises('paises.csv')
260    menu()
261    cargar_paises('paises.csv')
262    menu()
263    cargar_paises('paises.csv')
264    menu()
265    cargar_paises('paises.csv')
266    menu()
267    cargar_paises('paises.csv')
268    menu()
269    cargar_paises('paises.csv')
270    menu()
271    cargar_paises('paises.csv')
272    menu()
273    cargar_paises('paises.csv')
274    menu()
275    cargar_paises('paises.csv')
276    menu()
277    cargar_paises('paises.csv')
278    menu()
279    cargar_paises('paises.csv')
280    menu()
281    cargar_paises('paises.csv')
282    menu()
283    cargar_paises('paises.csv')
284    menu()
285    cargar_paises('paises.csv')
286    menu()
287    cargar_paises('paises.csv')
288    menu()
289    cargar_paises('paises.csv')
290    menu()
291    cargar_paises('paises.csv')
292    menu()
293    cargar_paises('paises.csv')
294    menu()
295    cargar_paises('paises.csv')
296    menu()
297    cargar_paises('paises.csv')
298    menu()
299    cargar_paises('paises.csv')
300    menu()
301    cargar_paises('paises.csv')
302    menu()
303    cargar_paises('paises.csv')
304    menu()
305    cargar_paises('paises.csv')
306    menu()
307    cargar_paises('paises.csv')
308    menu()
309    cargar_paises('paises.csv')
310    menu()
311    cargar_paises('paises.csv')
312    menu()
313    cargar_paises('paises.csv')
314    menu()
315    cargar_paises('paises.csv')
316    menu()
317    cargar_paises('paises.csv')
318    menu()
319    cargar_paises('paises.csv')
320    menu()
321    cargar_paises('paises.csv')
322    menu()
323    cargar_paises('paises.csv')
324    menu()
325    cargar_paises('paises.csv')
326    menu()
327    cargar_paises('paises.csv')
328    menu()
329    cargar_paises('paises.csv')
330    menu()
331    cargar_paises('paises.csv')
332    menu()
333    cargar_paises('paises.csv')
334    menu()
335    cargar_paises('paises.csv')
336    menu()
337    cargar_paises('paises.csv')
338    menu()
339    cargar_paises('paises.csv')
340    menu()
341    cargar_paises('paises.csv')
342    menu()
343    cargar_paises('paises.csv')
344    menu()
345    cargar_paises('paises.csv')
346    menu()
347    cargar_paises('paises.csv')
348    menu()
349    cargar_paises('paises.csv')
350    menu()
351    cargar_paises('paises.csv')
352    menu()
353    cargar_paises('paises.csv')
354    menu()
355    cargar_paises('paises.csv')
356    menu()
357    cargar_paises('paises.csv')
358    menu()
359    cargar_paises('paises.csv')
360    menu()
361    cargar_paises('paises.csv')
362    menu()
363    cargar_paises('paises.csv')
364    menu()
365    cargar_paises('paises.csv')
366    menu()
367    cargar_paises('paises.csv')
368    menu()
369    cargar_paises('paises.csv')
370    menu()
371    cargar_paises('paises.csv')
372    menu()
373    cargar_paises('paises.csv')
374    menu()
375    cargar_paises('paises.csv')
376    menu()
377    cargar_paises('paises.csv')
378    menu()
379    cargar_paises('paises.csv')
380    menu()
381    cargar_paises('paises.csv')
382    menu()
383    cargar_paises('paises.csv')
384    menu()
385    cargar_paises('paises.csv')
386    menu()
387    cargar_paises('paises.csv')
388    menu()
389    cargar_paises('paises.csv')
390    menu()
391    cargar_paises('paises.csv')
392    menu()
393    cargar_paises('paises.csv')
394    menu()
395    cargar_paises('paises.csv')
396    menu()
397    cargar_paises('paises.csv')
398    menu()
399    cargar_paises('paises.csv')
40
```

The image shows a Python IDE interface with a file named `Villegas_Ruiz_TPI_ProgramacionI.py` open. The script defines a function `cargar_paises(nombre_archivo)` that reads a CSV file and prints a warning if any fields are empty. The terminal window shows the output of the script, including a menu of options and a list of countries with their population and area.

```
3
4 def cargar_paises(nombre_archivo):
5     paises = []
6     try:
7         with open(nombre_archivo, "r", encoding="latin-1") as archivo:
8             lector = csv.DictReader(archivo)
9             for fila in lector:
10                 if not fila["nombre"] or not fila["poblacion"] or not fila["superficie"] or
11                    print("Aviso: registro con campos vacíos ignorado")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

6 - Filtrar por superficie
7 - Ordenar por nombre
8 - Ordenar por población
9 - Ordenar por superficie
10 - Estadísticas
0 - Salir

Opción: 6
Rango mínimo de superficie (km²): 15000
Rango máximo de superficie (km²): 800000

```
{'nombre': 'Japon', 'poblacion': 125800000, 'superficie': 377975, 'continente': 'Asia'}
{'nombre': 'Alemania', 'poblacion': 83149300, 'superficie': 357022, 'continente': 'Europa'}
{'nombre': 'Francia', 'poblacion': 67430000, 'superficie': 543965, 'continente': 'Europa'}
{'nombre': 'Nueva Zelanda', 'poblacion': 5080000, 'superficie': 268021, 'continente': 'Oceania'}
{'nombre': 'Italia', 'poblacion': 59270000, 'superficie': 301340, 'continente': 'Europa'}
{'nombre': 'Fiji', 'poblacion': 900000, 'superficie': 18272, 'continente': 'Oceania'}
{'nombre': 'Chile', 'poblacion': 19600000, 'superficie': 756102, 'continente': 'America'}
```

1 - Agregar país

Ln 153, Col 16 Spaces: 4 UTF-8 CRLF Python 3.14 (64-bit)

5. METODOLOGÍA UTILIZADA

Se desarrolló una aplicación modular utilizando funciones independientes para cada operación.

Los datos fueron cargados desde un archivo CSV mediante el módulo csv.

Se implementaron validaciones para evitar errores de ingreso y manejo de excepciones utilizando bloques try-except.

6. RESULTADOS OBTENIDOS

El sistema permite:

- Agregar países.
- Actualizar población y superficie.
- Buscar países por coincidencia parcial.
- Filtrar por continente.
- Filtrar por rango de población.
- Filtrar por rango de superficie.
- Ordenar por nombre.
- Ordenar por población.
- Ordenar por superficie.
- Obtener estadísticas generales.

Las pruebas realizadas demostraron un funcionamiento correcto de todas las funcionalidades solicitadas.

(Agregar aquí capturas de pantalla del programa funcionando)

7. DIFICULTADES ENCONTRADAS

Durante el desarrollo surgieron dificultades relacionadas con la validación de datos ingresados por el usuario y la lectura correcta de información desde archivos CSV.

También fue necesario implementar controles para evitar errores cuando las búsquedas no devolvían resultados o cuando se ingresaban valores inválidos.

Estas situaciones fueron resueltas mediante validaciones y estructuras de manejo de excepciones.

8. CONCLUSIONES

La realización del trabajo permitió aplicar de forma práctica los conceptos fundamentales de Programación I.

Se reforzó el uso de listas, diccionarios, funciones, estructuras condicionales y repetitivas, además de la lectura de archivos CSV y el procesamiento de información.

La experiencia permitió comprender la importancia de la modularización del código y de las validaciones para desarrollar programas más robustos y mantenibles.

9. BIBLIOGRAFÍA

Python Software Foundation.

Python Documentation.

<https://docs.python.org/3/>

Python Software Foundation.

Data Structures.

<https://docs.python.org/3/tutorial/datastructures.html>

Python Software Foundation.

CSV Module.

<https://docs.python.org/3/library/csv.html>

Downey, A.

Think Python: How to Think Like a Computer Scientist.

Sweigart, A.
Automate the Boring Stuff with Python.